

上海财经大学

信息管理与工程学院

梯度下降方法

优化理论与算法第六章

王震

2026.4.30



目录

应用动机：推荐系统与矩阵补全

协同过滤与矩阵分解

无约束优化的迭代方法

下降方向与步长

收敛速度与梯度下降

梯度下降与二次函数特例

光滑强凸情形



本章路线

- ▶ 从推荐系统中的缺失评分预测出发，引出大规模最小二乘与矩阵分解模型。
- ▶ 将这些模型抽象为无约束优化问题，讨论迭代格式、下降方向与步长选择。
- ▶ 以梯度下降和最速下降为主线，理解收敛度量、收敛速度以及光滑凸优化中的典型结论。



猜猜缺失的数字

14, 2, ?, 6, 3, 1, 21, 7, 9

你能找出问号处应该是什么数字吗？



再猜一次

1 2 3

? 6 9

7 14 21

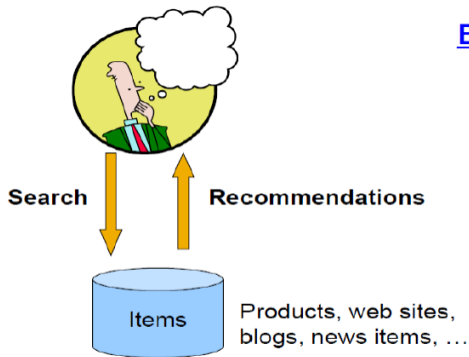
这次缺失的数字是什么呢？



	Alice	Bob	Chris	David
X-Men	3	5	4	4
Yellow Rock	2	3	?	3
Zoolander	4	4	4	?

你能补全缺失的评分吗？

动机示例：推荐系统



Examples:



这些系统如何预测你的偏好?



Netflix Prize: 为什么推荐系统成为优化问题

竞赛背景

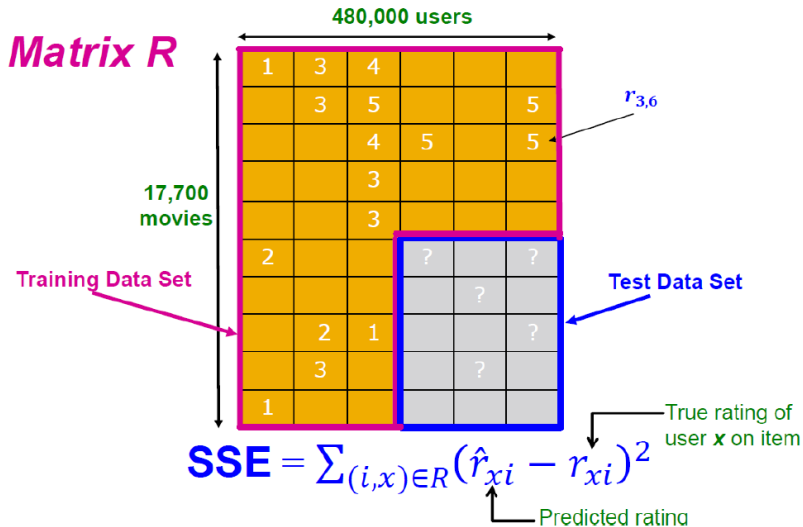
- ▶ Netflix 于 2006 年发起比赛，目标是将 Cinematch 的 RMSE 从 0.9514 降低 10%。
- ▶ 大奖门槛：隐藏测试集 RMSE 不超过约 0.8572；奖金 100 万美元。
- ▶ 比赛持续近三年，全球超过 5 万名参赛者参与。

数据规模

- ▶ 训练集：约 1 亿条评分，48 万用户，17,770 部电影，覆盖 2000–2005 年。
- ▶ 公开反馈与最终评测分属不同集合；评估标准为均方根误差（RMSE）：
$$\sqrt{\frac{1}{|\Omega_{\text{test}}|} \sum_{(x,i) \in \Omega_{\text{test}}} (\hat{r}_{xi} - r_{xi})^2}$$



Netflix 评估方法





公开榜不等于最终榜

- ▶ 排行榜显示的是 **quiz set** 成绩，真正决定大奖的是隐藏 **test set** 成绩。
- ▶ 公开榜领先不一定代表泛化更好：频繁根据公开反馈调参，可能形成对 **quiz set** 的过拟合（teaching to the test）。
- ▶ 若隐藏 **test set** 上四位小数 **RMSE** 打平，规则按提交时间先后决定冠军。

因此，这场比赛不仅考算法，也考验证集设计、泛化能力和竞赛策略。



目录

应用动机：推荐系统与矩阵补全

协同过滤与矩阵分解

无约束优化的迭代方法

下降方向与步长

收敛速度与梯度下降

梯度下降与二次函数特例

光滑强凸情形



推荐系统中的记号

- ▶ 用 x 表示用户, i, j 表示电影; 评分矩阵 $R = (r_{xi})$ 通常只在少量位置有观测值。
- ▶ Ω_{train} 表示训练集中已知评分的用户--电影对, $\Omega_i = \{x : (x, i) \in \Omega_{\text{train}}\}$ 。
- ▶ 推荐模型的核心任务是: 只利用 Ω_{train} 上的评分, 预测未观测位置的 $\hat{r}_{xi}$ 。



推荐系统的几条建模路线

- ▶ 路线一：邻域模型。如果两部电影被相似用户以相似方式评分，就用相似电影的评分残差预测目标电影。
- ▶ 路线二：矩阵分解。假设用户偏好和电影属性可由少数隐因子解释，用低维向量内积补全缺失评分。
- ▶ 扩展路线：偏差、时间动态与融合。真实评分还包含用户习惯、电影热度、时间变化以及多模型互补信息。

这些路线最后都会落到“构造损失函数并用迭代算法求解”的优化问题。

Netflix Prize: 从算法比赛到团队融合

竞赛概况

- ▶ 参赛团队：2700+，参赛者超过 5 万。
- ▶ 目标：在 Cinematch 基础上降低 10% RMSE，而不是简单提高分类准确率。
- ▶ 后期领先团队的共同特征：大量模型的融合（ensemble / blend）。



路线一：先分清 i 、 j 和 $N(i; x)$

- ▶ x : 当前用户。
- ▶ i : 目标电影，通常是用户 x 尚未评分、我们想预测的电影。
- ▶ j : 邻居电影，必须是用户 x 已经评分过的电影，才能使用真实评分 r_{xj} 。
- ▶ $N(i; x)$: 用户 x 已评分电影中，与目标电影 i 最相似的一小部分电影：

$$N(i; x) = \{j : (x, j) \in \Omega_{\text{train}}, j \text{ 与 } i \text{ 相似}\}.$$

- ▶ 实际计算时通常只取相似度最高的前 k 个邻居，避免邻域过大。

路线一：邻域模型（加权协同过滤）

$$\hat{r}_{xi} = b_{xi} + \sum_{j \in N(i;x)} w_{ij}(r_{xj} - b_{xj})$$

► 基线估计：

$$b_{xi} = \mu + b_x + b_i$$

► μ : 全局平均评分

► b_x : 用户 x 的评分偏差 = (用户 x 的平均评分) - μ

► b_i : 电影 i 的评分偏差 = (电影 i 的平均评分) - μ

► 公式的结构：

► 先用 b_{xi} 给出“不看邻居电影时”的基础预测；

► 再用用户 x 对相似电影 j 的评分残差来修正目标电影 i 的预测。

路线一：为什么使用评分残差？

$$r_{xj} - b_{xj}$$

- ▶ b_{xj} 是对用户 x 给电影 j 打分的基线预测，已经包含全局平均、用户打分习惯和电影热度。
- ▶ $r_{xj} - b_{xj}$ 表示用户 x 对电影 j 的“超出预期”的喜欢或不喜欢。
- ▶ 若用户本来就爱打高分，或电影 j 本来就很热门，直接使用 r_{xj} 会重复计算这些因素。
- ▶ 因此邻域模型加权的是残差：相似电影 j 上的额外偏好，能在多大程度上转移到目标电影 i 。

一句话： b_{xi} 管“平均和偏差”，残差项管“相似电影带来的个性化修正”。

路线一：如何学习相似电影的权重

- ▶ 对每部目标电影 i , w_{ij} 是电影 j 对电影 i 的插值权重（实数），允许：

$$\sum_{j \in N(i; x)} w_{ij} \neq 1$$

- ▶ w_{ij} 建模电影对之间的交互，通常与用户 x 无关。
- ▶ 如果对所有电影对都学习 w_{ij} ，参数规模约为“电影数的平方”，会非常大。
- ▶ 实际做法：先为每部电影 i 保留相似度最高的前 k 个候选邻居 $\mathcal{S}_i$ ，再令

$$N(i; x) = \mathcal{S}_i \cap \{j : (x, j) \in \Omega_{\text{train}}\}.$$

- ▶ 这样每次预测只使用用户 x 已评分、且与目标电影 i 足够相似的少量电影。
- ▶ 通过训练集中已知的电影 i 的评分优化权重：

$$\min_{\{w_{ij}\}} F_i(w) := \sum_{x \in \Omega_i} \left(b_{xi} + \sum_{j \in N(i; x)} w_{ij} (r_{xj} - b_{xj}) - r_{xi} \right)^2$$

路线一：从正规方程到迭代法

- 这是关于权重 w 的无约束二次优化问题。令

$$e_{xi}(w) = b_{xi} + \sum_{k \in N(i;x)} w_{ik}(r_{xk} - b_{xk}) - r_{xi},$$

则对所有出现在邻域中的 j ，一阶条件为

$$\frac{\partial F_i(w)}{\partial w_{ij}} = 2 \sum_{x \in \Omega_i: j \in N(i;x)} e_{xi}(w)(r_{xj} - b_{xj}) = 0.$$

- 因为目标函数是二次函数，一阶条件等价于正规方程；在大规模稀疏数据中通常采用迭代法近似求解。
- 梯度下降法：

$$w^{k+1} = w^k - \tau \nabla F_i(w^k)$$

- 小结：邻域模型直观、可解释，但电影对权重较多；这正是后续矩阵分解用低维隐向量压缩参数的动机。

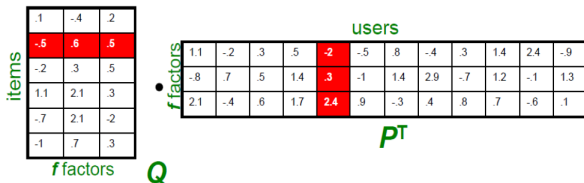
路线二：矩阵分解（隐因子模型）

- 如何估计用户 x 对项目 i 的缺失评分？

$$\hat{r}_{xi} = q_i^T p_x = \sum_{f=1}^r q_{if} p_{xf}$$

其中 $q_i \in \mathbb{R}^r$ 是电影 i 的隐向量， $p_x \in \mathbb{R}^r$ 是用户 x 的隐向量。

- r 是隐因子维度（latent dimension / embedding dimension），是预先选择的超参数，而不是由数据唯一决定。
- 通常取 $r \ll \min\{\text{用户数}, \text{电影数}\}$ ；实际可在验证集上从几十到几百维中选择。
- 直观理解： r 越大表达能力越强，但计算成本和过拟合风险也更高。



路线二：不完全评分矩阵优化

不完全评分矩阵优化

- ▶ 评分矩阵 R 存在缺失值！需在训练数据上最小化 SSE:

$$\min_{P, Q} \sum_{(x, i) \in \Omega_{\text{train}}} (r_{xi} - q_i^T p_x)^2$$

- ▶ 若 P 的第 x 行为 p_x^T , Q 的第 i 行为 q_i^T , 则预测矩阵可写为

$$\hat{R} = PQ^T, \quad \text{rank}(\hat{R}) \leq r.$$

- ▶ 因此, 选择 r 已经是在控制预测矩阵的秩上界; 训练时只在观测集合 Ω_{train} 上拟合。

路线二：正则化目标函数

正则化目标函数

$$\min_{P, Q} \sum_{(x, i) \in \Omega_{\text{train}}} (r_{xi} - q_i^T p_x)^2 + \lambda \left[\sum_x \|p_x\|_2^2 + \sum_i \|q_i\|_2^2 \right]$$

- ▶ 正则项是所有用户隐向量和电影隐向量的平方二范数之和：

$$\sum_x \|p_x\|_2^2 + \sum_i \|q_i\|_2^2 = \|P\|_F^2 + \|Q\|_F^2.$$

- ▶ 它惩罚过大的参数，减少模型为了拟合少量观测评分而过拟合的风险。
- ▶ 它也缓解分解的尺度不唯一性： $q_i^T p_x = (cq_i)^T (p_x/c)$ ，预测不变但参数大小可能失控。

路线二：为什么不直接正则化 **rank**?

- ▶ 在矩阵分解中，隐因子维度 r 已经给出

$$\text{rank}(\hat{R}) = \text{rank}(PQ^T) \leq r.$$

- ▶ 因此，选择 r 本身就是对秩的结构性控制；正则项主要控制参数大小。
- ▶ 直接惩罚 $\text{rank}(\hat{R})$ 并不适合这里的梯度法：
 - ▶ **rank** 是离散、非光滑、非凸的；
 - ▶ 训练变量是 P, Q ，不是完整矩阵 $\hat{R}$ 的每个元素；
 - ▶ 大规模稀疏数据更适合用 r 加 **Frobenius** 正则的方式。

路线二：用梯度下降学习隐向量

► 梯度下降步骤：

► 初始化 P 和 Q （例如随机小值；缺失评分不进入损失函数）

► 迭代更新：

$$P^{k+1} \leftarrow P^k - \tau \nabla_P F(P^k, Q^k),$$

$$Q^{k+1} \leftarrow Q^k - \tau \nabla_Q F(P^k, Q^k).$$

$$(\nabla_{q_i} F)_f = -2 \sum_{x \in \Omega_i} (r_{xi} - q_i^T p_x) p_{xf} + 2\lambda q_{if},$$

$$(\nabla_{p_x} F)_f = -2 \sum_{i: (x, i) \in \Omega_{\text{train}}} (r_{xi} - q_i^T p_x) q_{if} + 2\lambda p_{xf}.$$

路线二：随机梯度下降

样本级可加，但变量不完全可分

$$F(P, Q) = \sum_{(x, i) \in \Omega_{\text{train}}} F_{xi}(P, Q), \quad F_{xi} = \frac{1}{2}(r_{xi} - \mathbf{q}_i^T \mathbf{p}_x)^2 + \frac{\lambda}{2}(\|\mathbf{p}_x\|_2^2 + \|\mathbf{q}_i\|_2^2).$$

- ▶ 这里的“可加”指损失可以按观测评分 (x, i) 分解为许多样本项。
- ▶ 但变量并不完全可分：同一个 $\mathbf{p}_x$ 会出现在用户 x 的所有评分中，同一个 $\mathbf{q}_i$ 会出现在电影 i 的所有评分中。
- ▶ SGD 的思想是：每次抽取一个观测评分，用它近似全梯度，而不是每步扫描全部 Ω_{train} 。

路线二：为什么 SGD 适合稀疏评分？

- ▶ 单个样本 (x, i) 只涉及两个局部参数：用户向量 p_x 和电影向量 q_i 。
- ▶ 令 $e_{xi} = r_{xi} - q_i^T p_x$ ，使用更新前的 p_x, q_i 计算单步更新：
 - ▶ $p_x \leftarrow p_x + \tau(e_{xi} q_i - \lambda p_x)$;
 - ▶ $q_i \leftarrow q_i + \tau(e_{xi} p_x - \lambda q_i)$ 。
- ▶ 其余用户向量和电影向量本步不更新，因此每步成本很低。
- ▶ 优势与代价：
 - ▶ 需要更多迭代步数
 - ▶ 每步计算速度显著加快
 - ▶ 特别适合大规模稀疏数据



扩展路线：偏差、时间动态与模型融合

- ▶ 偏差项：用户有人偏严、有人偏宽；电影也有整体受欢迎程度差异。
- ▶ 时间动态：用户兴趣会变化，电影热度也会随上映时间和讨论热度变化。
- ▶ 模型融合：邻域模型、矩阵分解、时间模型等各有盲点，融合可以利用互补误差。



扩展路线：加入偏差项

► 预测模型：

$$\hat{r}_{xi} = \mu + b_x + b_i + q_i^T p_x$$

► μ ：总体平均评分， b_x ：用户 x 的偏差， b_i ：电影 i 的偏差。

► 新模型：

$$\min_{P, Q, \{b_x\}, \{b_i\}} \sum_{(x, i) \in \Omega_{\text{train}}} \left(r_{xi} - (\mu + b_x + b_i + q_i^T p_x) \right)^2 + \lambda \left[\sum_x \|p_x\|_2^2 + \sum_i \|q_i\|_2^2 + \sum_x b_x^2 + \sum_i b_i^2 \right]$$



扩展路线：添加时间依赖性

- ▶ 电影年龄：
 - ▶ 用户无理由偏好新电影
 - ▶ 老电影本质上比新电影更好
- ▶ 将时间依赖性添加到偏差中：

$$\hat{r}_{xi} = \mu + b_x(t) + b_i(t) + q_i^T p_x$$

- ▶ 使参数 b_x 和 b_i 依赖于时间
- ▶ 每个区间对应连续 10 周 $b_i(t) = b_i + b_{i,\text{Bin}(t)}$

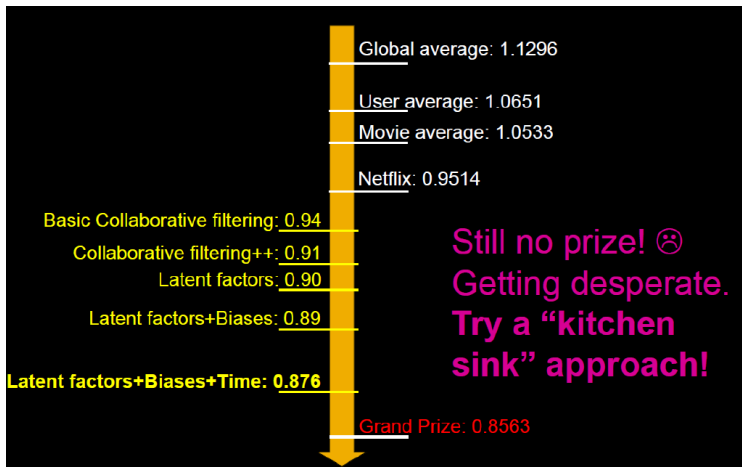


推荐系统案例小结

- ▶ 加权协同过滤、矩阵分解和带偏差模型都可以写成最小化训练误差的优化问题。
- ▶ 数据规模大且评分矩阵稀疏，使得直接求精确解往往代价很高。
- ▶ 因此，实际算法通常依赖梯度下降、随机梯度下降、线搜索和正则化等基本工具。

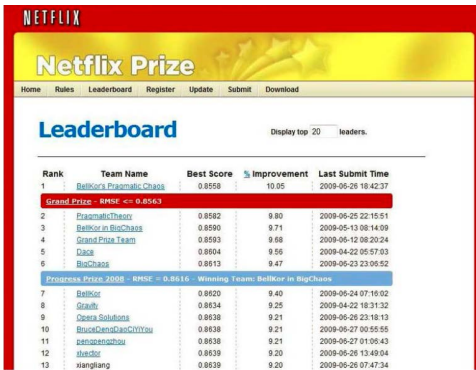


扩展路线：模型融合的效果



2009 年 6 月 26 日: Last Call 开始

- ▶ BellKor 与 BigChaos、Pragmatic Theory 合并后, 形成 BellKor's Pragmatic Chaos。
- ▶ 2009 年 6 月 26 日, 他们率先突破 10% 改进门槛, 触发最后 30 天的 **last call**。
- ▶ 这不是比赛结束, 而是把其他强队推向更激烈的模型融合与团队合并。



Rank	Team Name	Best Score	% Improvement	Last Submit Time
1	BellKor's Pragmatic Chaos	0.8558	10.05	2009-06-26 18:42:37
Grand Prize - RMSE $\leq$ 0.8563				
2	PragmaticTheory	0.8582	9.80	2009-06-25 22:15:51
3	BellKor in BigChaos	0.8590	9.71	2009-05-13 08:14:09
4	Grand Prize Team	0.8593	9.68	2009-06-12 08:20:24
5	Dade	0.8604	9.56	2009-04-22 05:57:03
6	BigChaos	0.8613	9.47	2009-06-23 23:06:52
Progress Prize 2008 - RMSE $\approx$ 0.8616 - Winning Team: BellKor in BigChaos				
7	BellKor	0.8620	9.40	2009-06-24 07:16:02
8	Grady	0.8634	9.25	2009-04-22 18:31:32
9	Opera Solutions	0.8638	9.21	2009-06-26 23:18:13
10	BruceDenoDanCrittYou	0.8638	9.21	2009-06-27 00:55:55
11	penapengzhou	0.8638	9.21	2009-06-27 01:06:43
12	thudior	0.8639	9.20	2009-06-26 13:49:04
13	xiangliang	0.8639	9.20	2009-06-26 07:47:34

June 26th submission triggers 30-day "last call"

最后 30 天：两个“大联盟”的竞争

▶ BellKor's Pragmatic Chaos

- ▶ 继续在已有模型上做小幅改进。
- ▶ 组合矩阵分解、邻域模型、偏差修正和时间动态等多个模块。

▶ The Ensemble

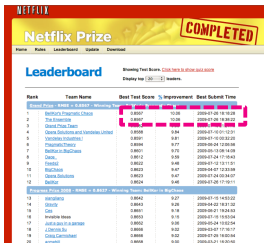
- ▶ 由 30 多个竞争者合并形成。
- ▶ 核心策略很直接：把很多团队的预测结果做加权融合（blend）。
- ▶ 说明后期最有效的不是单一“神奇模型”，而是多样化模型的互补。

最后 24 小时：提交时间也成为策略

- ▶ 最后 24 小时只能做一次最终提交；双方都盯着公开 leaderboard。
- ▶ 公开榜只反映 quiz set，不等于隐藏 test set 的最终排名。
- ▶ BellKor 有意提前提交，约在截止前 40 分钟；The Ensemble 约 20 分钟后提交最终条目。
- ▶ 这是一种理性策略：
 - ▶ 避免最后几分钟上传失败或服务器拥堵；
 - ▶ 若隐藏 test set 打平，提交时间就是 tie-breaker。

最终结果：公开榜与隐藏榜不一致

- ▶ 公开 quiz leaderboard: The Ensemble 约为 0.8553, BellKor 约为 0.8554, 看起来 The Ensemble 更好。
- ▶ 隐藏 test set: 两队四位小数均为 0.8567。
- ▶ 更精确地看: BellKor 约为 0.856704, The Ensemble 约为 0.856714。
- ▶ 按规则四位小数打平, 于是由提交顺序决定冠军: BellKor 获胜。



Rank	Team Name	Best Test Score	% Improvement	Best Submit Time
1	BellKor, Crayon's Colors	0.8554	10.5%	2009-07-08 16:18:28
2	The Ensemble	0.8553	10.5%	2009-07-08 18:38:22
3	Qora's Data Team	0.8551	9.9%	2009-06-24 12:05:58
4	Qora Solutions and Ventures Limited	0.8550	9.8%	2009-07-01 01:12:31
5	VortexData Solutions	0.8551	9.8%	2009-07-01 00:32:20
6	Pharmacia Therapies	0.8550	9.7%	2009-06-24 12:05:58
7	BellKor in BlueCross	0.8551	9.7%	2009-06-23 08:14:09
8	Qora	0.8552	9.5%	2009-07-04 17:16:03
9	Ensemble	0.8552	9.4%	2009-07-02 13:11:51
10	Pharmacia	0.8553	9.4%	2009-06-07 12:33:08
11	Qora Solutions	0.8553	9.4%	2009-07-24 00:34:07
12	BellKor	0.8554	9.4%	2009-07-08 17:19:11
13	Pharmacia	0.8542	9.2%	2009-07-01 14:53:22
14	Qora	0.8543	9.2%	2009-06-22 18:31:02
15	Qora	0.8551	9.1%	2009-06-27 14:24:03
16	Ensemble Ideas	0.8553	9.1%	2009-07-01 18:53:04
17	Qora's Data Team	0.8552	9.0%	2009-06-24 14:02:04
18	J. DeWitt, Inc.	0.8558	9.0%	2009-03-07 17:16:17
19	Qora, Cambridge	0.8558	9.0%	2009-07-01 18:00:04
20	Qora	0.8558	9.0%	2009-03-27 16:00:00



这个故事说明什么？

- ▶ 公开验证集领先不等于真实泛化更好。The Ensemble 赢了公开榜，但隐藏 test set 没有拉开差距。
- ▶ 模型融合非常重要。最终强队都依赖 ensemble / blend，很多小模型的边际贡献会在融合中体现出来。
- ▶ 竞赛规则会改变算法策略。当 tie-breaker 是提交时间时，“什么时候提交”也变成优化决策的一部分。
- ▶ 回到本章主线：推荐系统中的模型选择、正则化、验证集设计和迭代求解，都是优化思想的具体体现。



目录

应用动机：推荐系统与矩阵补全

协同过滤与矩阵分解

无约束优化的迭代方法

下降方向与步长

收敛速度与梯度下降

梯度下降与二次函数特例

光滑强凸情形



从应用回到算法

- ▶ 前面的推荐系统模型说明：复杂应用最终常落到一个形如

$$\min_x f(x)$$

的无约束优化问题。

- ▶ 接下来关注三个基本问题：沿什么方向走、走多远、怎样判断算法收敛得快不快。
- ▶ 梯度下降是最基本的例子；牛顿法、拟牛顿法和随机梯度法都可以看作对方向或梯度信息的不同利用方式。



目录

应用动机：推荐系统与矩阵补全

协同过滤与矩阵分解

无约束优化的迭代方法

下降方向与步长

收敛速度与梯度下降

梯度下降与二次函数特例

光滑强凸情形



迭代方法

► 基本迭代格式:

$$x^{k+1} = x^k + \alpha_k d^k,$$

其中 d^k 是搜索方向, $\alpha_k > 0$ 是步长。

- 若存在足够小的 $\alpha > 0$ 使 $f(x + \alpha d) < f(x)$, 则 d 是 f 在 x 处的下降方向。
- 当 f 可微且 $\nabla f(x)^T d < 0$ 时, d 是下降方向; 等价地, d 与 $-\nabla f(x)$ 的夹角小于 90° 。

► 一些下降方向的选择:

- 梯度下降方向: $d^k := -\nabla f(x^k)$ (若 $\nabla f(x^k) \neq 0$)
- 牛顿方向: $d^k := -(\nabla^2 f(x^k))^{-1} \nabla f(x^k)$, 通常要求 Hessian 可逆; 若 $\nabla^2 f(x^k) \succ 0$, 它是下降方向。
- 拟牛顿方向: $d^k := -H_k \nabla f(x^k)$, 其中 $H_k \succ 0$ 。

确定步长

- 精确最小化：找到 $\alpha_k > 0$ 使得

$$f(x^k + \alpha_k d^k) = \min_{\alpha \geq 0} f(x^k + \alpha d^k)$$

这可以通过线搜索实现

- 二分法
- 黄金分割法
- Armijo 规则：固定 $s > 0, \beta \in (0, 1)$ 和 $\sigma \in (0, 1)$ 。令 $m \geq 0$ 为满足以下条件的最小整数

$$f(x^k) - f(x^k + \beta^m s d^k) \geq -\sigma \beta^m s \nabla f(x^k)^T d^k$$

并令 $\alpha_k = s\beta^m$ 。这里要求 d^k 是下降方向，因此 $\nabla f(x^k)^T d^k < 0$ 。

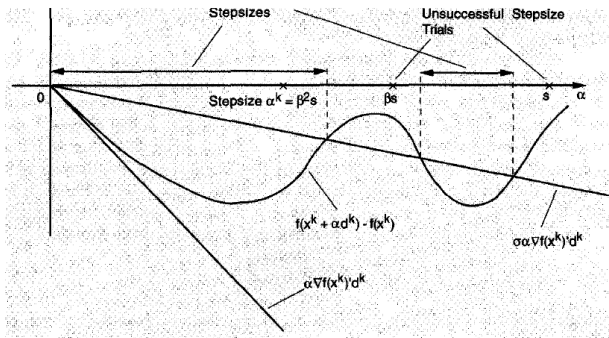
- 递减步长： $\alpha_k \downarrow 0$ 且 $\sum_{k=1}^{\infty} \alpha_k = \infty$ 。
- 常数步长，即 $\alpha_k \equiv s > 0$ 。

Armijo 规则的线搜索

- ▶ 先从候选步长 s 开始试探；若下降不够明显，就依次缩短为 $s\beta, s\beta^2, \dots$ 。
- ▶ Armijo 条件要求新点的函数值低于一条“允许的下降直线”：

$$f(x^k + \alpha d^k) \leq f(x^k) + \sigma \alpha \nabla f(x^k)^T d^k.$$

- ▶ 它不追求一维方向上的最优步长，只要求“下降足够多”，因此比精确线搜索便宜。





收敛的基本概念

- ▶ 可能的误差测量：
 - ▶ $e(x^k) = \|x^k - x^*\|$;
 - ▶ $e(x^k) = f(x^k) - f(x^*)$;
 - ▶ $e(x^k) = \|\nabla f(x^k)\|$ 。
- ▶ 收敛性： $\lim_{k \rightarrow \infty} x^k = x^*$ 或 $\lim_{k \rightarrow \infty} f(x^k) = f(x^*)$ 。
- ▶ 迭代复杂度：常见结论包括 $O(1/k)$ 、 $O(1/k^2)$ 、 $O(1/\sqrt{k})$ 等，具体速度取决于凸性、光滑性和步长规则。



目录

应用动机：推荐系统与矩阵补全

协同过滤与矩阵分解

无约束优化的迭代方法

下降方向与步长

收敛速度与梯度下降

梯度下降与二次函数特例

光滑强凸情形

线性收敛中的“线性”是什么意思？

- ▶ 这里的“线性”不是说误差像直线一样下降，而是说相邻两步误差的比值趋于一个常数：

$$\frac{e_{k+1}}{e_k} \rightarrow \mu, \quad 0 < \mu < 1.$$

- ▶ 因而远离初始阶段后，大致有

$$e_k \approx C\mu^k.$$

- ▶ 也就是说，每迭代一步，误差大约缩小为原来的 μ 倍。例如 $\mu = 0.5$ 表示每步约减半。
- ▶ 这种速度也常称为几何收敛（geometric convergence）。

渐近收敛速度

- ▶ 假设序列 $\{x^k\}$ 收敛到 $\bar{x}$ 。执行比率测试

$$\lim_{k \rightarrow \infty} \frac{e(x^{k+1})}{e(x^k)} = \mu,$$

其中 $e(x^k)$ 可以是 $\|x^k - x^*\|$ 、 $|f(x^k) - f(x^*)|$ 、 $\|\nabla f(x^k)\|$ 等非负误差。

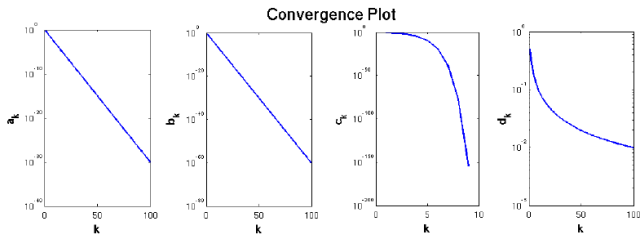
- ▶ 如果 $\mu \in (0, 1)$ ，则 $\{x^k\}$ 线性收敛。
 - ▶ 如果 $\mu = 0$ ，则 $\{x^k\}$ 超线性收敛。
 - ▶ 如果 $\mu = 1$ ，通常不能判定为线性收敛；常见情形是次线性收敛，需要进一步比较 $e(x^k)$ 与 $1/k^p$ 等序列。
- ▶ 为了区分超线性收敛速度，我们检查

$$\lim_{k \rightarrow \infty} \frac{e(x^{k+1})}{e(x^k)^q} = \mu > 0$$

若 $q > 1$ 且 $0 < \mu < \infty$ ，称为 q 阶收敛；特别地， $q = 2$ 时为二次收敛。

示例：这些序列对应什么收敛速度？

- ▶ $a_k = 1/2^k$: $\frac{a_{k+1}}{a_k} \rightarrow \frac{1}{2}$, 线性收敛, $\mu = \frac{1}{2}$ 。
- ▶ $b_k = 1/4^{\lfloor k/2 \rfloor}$: 比值在 1 和 $\frac{1}{4}$ 之间振荡, 不能用简单极限 μ 描述, 但总体仍呈几何下降。
- ▶ $c_k = 1/2^{2^k}$: $\frac{c_{k+1}}{c_k} \rightarrow 0$, 超线性收敛, 下降非常快。
- ▶ $d_k = 1/(k+1)$: $\frac{d_{k+1}}{d_k} \rightarrow 1$, 典型次线性收敛, 需要看 $O(1/k)$ 这类复杂度。





从 k 到 ϵ : 复杂度怎么换算?

- ▶ 如果误差满足 $e_k \leq C\mu^k$, 其中 $0 < \mu < 1$, 要达到 $e_k \leq \epsilon$, 只需

$$k \geq \frac{\log(C/\epsilon)}{\log(1/\mu)}.$$

因此线性收敛对应 $O(\log(1/\epsilon))$ 次迭代。

- ▶ 如果目标函数误差满足 $f(x^k) - f(x^*) \leq C/k$, 要达到 ϵ 精度, 需要

$$k \geq C/\epsilon,$$

即 $O(1/\epsilon)$ 。

- ▶ 如果 $f(x^k) - f(x^*) \leq C/k^2$, 则需要 $k \geq \sqrt{C/\epsilon}$, 即 $O(1/\sqrt{\epsilon})$ 。
- ▶ 所以“关于 k 的收敛率”和“达到 ϵ 精度的迭代复杂度”可以互相换算。



目录

应用动机：推荐系统与矩阵补全

协同过滤与矩阵分解

无约束优化的迭代方法

下降方向与步长

收敛速度与梯度下降

梯度下降与二次函数特例

光滑强凸情形



梯度下降算法

- ▶ 从当前点 x^k ，通过

$$x^{k+1} = x^k - \alpha_k \nabla f(x^k)$$

生成下一个点 x^{k+1} 。

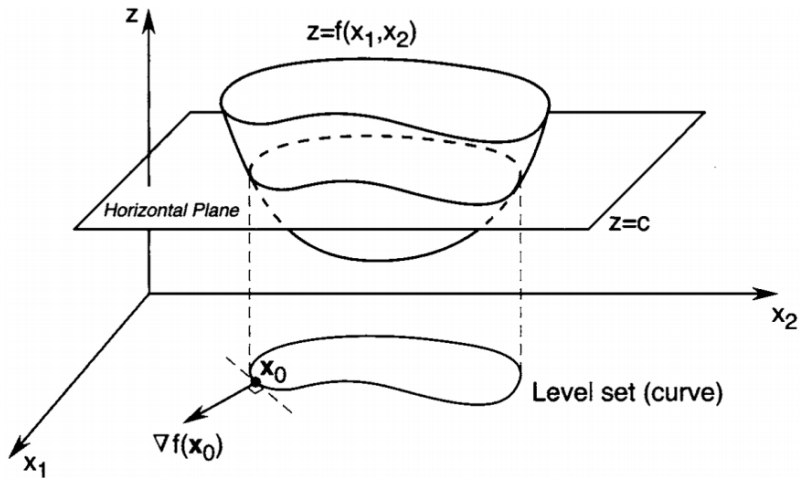
- ▶ 主要特点

- ▶ 最受欢迎的方法（在连续优化中）
- ▶ 简单直观，工作假设很少
- ▶ 可以与其他许多方法结合使用：对偶性、分裂、坐标下降、交替方向、随机、在线等。
- ▶ 适合大规模问题，例如，对于目标中有许多项的问题易于并行化。



梯度

- $\nabla f(\mathbf{x}_0)$ 垂直于通过 $\mathbf{x}_0$ 的 f 的水平集曲线的切线，指向函数值增加最快的方向。

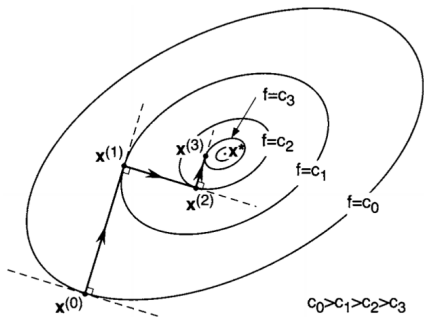


最速下降法

- 这是具有精确线搜索的梯度下降。即步长由精确最小化确定

$$\alpha_k = \arg \min_{\alpha \geq 0} f(x^k - \alpha \nabla f(x^k))$$

- 对二次函数可以得到闭式步长；对一般函数，精确线搜索往往比回溯线搜索更昂贵。





二次规划的最速下降

- ▶ 假设 Q 是对称正定矩阵，并考虑二次规划

$$f(x) = \frac{1}{2}x^T Qx - b^T x$$

其中 $\nabla f(x) = Qx - b$ 。

- ▶ 最速下降迭代：从任意 $x^{(0)}$ 开始，设置

$$x^{(k+1)} = x^{(k)} - \alpha_k g^{(k)}, \quad k = 0, 1, 2, \dots$$

其中 $g^{(k)} := \nabla f(x^{(k)})$ 且

$$\alpha_k = \arg \min_{\alpha \geq 0} f(x^{(k)} - \alpha g^{(k)}) = \frac{g^{(k)T} g^{(k)}}{g^{(k)T} Q g^{(k)}}$$



最速下降法的性能

- ▶ 每次迭代成本：主要由两个矩阵-向量乘法主导：
 - ▶ $g^{(k)} = Qx^{(k)} - b$
 - ▶ 计算 α_k 涉及 $Qg^{(k)}$
- ▶ 收敛速度：由初始点和 Q 的谱条件数决定。为了分析它们，我们
 - ▶ 定义解误差： $e^{(k)} = x^{(k)} - x^*$
 - ▶ 具有性质： $g^{(k)} = Qx^{(k)} - b = Qe^{(k)}$ 。



好的情况

- ▶ 若误差方向 $\mathbf{e}^{(k)}$ 正好是 $\mathbf{Q}$ 的特征向量，特征值为 λ ，则负梯度方向刚好指向最优点。

$$\begin{aligned}\mathbf{e}^{(k+1)} &= \mathbf{e}^{(k)} - \alpha_k \mathbf{g}^{(k)} = \mathbf{e}^{(k)} - \frac{\mathbf{g}^{(k)T} \mathbf{g}^{(k)}}{\mathbf{g}^{(k)T} \mathbf{Q} \mathbf{g}^{(k)}} (\mathbf{Q} \mathbf{e}^{(k)}) \\ &= \mathbf{e}^{(k)} + \frac{\mathbf{g}^{(k)T} \mathbf{g}^{(k)}}{\lambda \mathbf{g}^{(k)T} \mathbf{g}^{(k)}} (-\lambda \mathbf{e}^{(k)}) = \mathbf{0}\end{aligned}$$

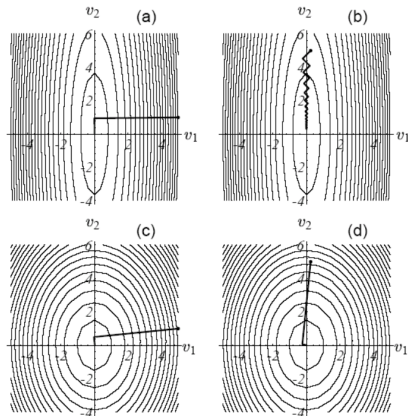
- ▶ 因而这一步就到达最优解。特别地，若 $\mathbf{Q}$ 只有一个不同特征值，水平集是圆，最速下降表现非常好。
- ▶ 一般情况下：定义 $\|\mathbf{e}\|_Q := \sqrt{\mathbf{e}^T \mathbf{Q} \mathbf{e}}$ 和条件数 $\kappa := \lambda_{\max}(\mathbf{Q})/\lambda_{\min}(\mathbf{Q})$ ，则

$$\|\mathbf{e}^{(k)}\|_Q \leq \left(\frac{\kappa - 1}{\kappa + 1} \right)^k \|\mathbf{e}^{(0)}\|_Q$$

- ▶ κ 越接近 1，收敛因子越小，下降越快； κ 越大，椭圆越狭长，容易出现之字形。

最速下降的图形理解

- ▶ 圆形水平集：负梯度方向直接朝向中心，精确线搜索可能一步到达最优点。
- ▶ 狭长椭圆水平集：每次都沿当前最陡方向下降，但下一步方向几乎转成另一个方向，形成之字形路径。
- ▶ 因此图中的慢收敛不是因为每步没有下降，而是因为曲率差异大，条件数 κ 大。





目录

应用动机：推荐系统与矩阵补全

协同过滤与矩阵分解

无约束优化的迭代方法

下降方向与步长

收敛速度与梯度下降

梯度下降与二次函数特例

光滑强凸情形

强凸函数的收敛性分析

- ▶ 假设 f 二阶可微, 且存在 $0 < m \leq M$ 使

$$mI \preceq \nabla^2 f(x) \preceq MI, \quad \forall x.$$

这表示 f 是 m -强凸且 M -光滑的。

- ▶ m 控制函数至少有多“弯”(下界曲率), M 控制函数最多有多“弯”(上界曲率)。
- ▶ 条件数 $\kappa = M/m$ 描述曲率差异; κ 越大, 水平集越狭长, 梯度下降越慢。
- ▶ 根据二阶泰勒展开 (Lagrange 余项) 有

$$f(x + \alpha d) = f(x) + \alpha \nabla f(x)^T d + \frac{\alpha^2}{2} d^T \nabla^2 f(\bar{x}) d,$$

其中 $\bar{x}$ 位于 x 和 $x + \alpha d$ 之间。

- ▶ 如果我们令 $\alpha = \frac{1}{M}$ 且 $d = -\nabla f(x)$, 则

$$f(x + \alpha d) \leq f(x) - \frac{1}{2M} \|\nabla f(x)\|^2$$

- ▶ 另一方面, 由强凸性的一阶下界可得

$$f(x) - f(x^*) \leq \|\nabla f(x)\| \cdot \|x - x^*\| - \frac{m}{2} \|x - x^*\|^2 \leq \frac{1}{2m} \|\nabla f(x)\|^2$$



进一步分析

- ▶ 将强凸性给出的误差界代入下降不等式，得到

$$f(x - \frac{1}{M} \nabla f(x)) \leq f(x) - \frac{1}{2M} \|\nabla f(x)\|^2 \leq f(x) - \frac{m}{M} (f(x) - f(x^*)).$$

- ▶ 因而固定步长 $\alpha = 1/M$ 的梯度下降满足目标函数误差的线性收敛：

$$f(x^+) - f(x^*) \leq \left(1 - \frac{m}{M}\right) (f(x) - f(x^*)), \quad x^+ = x - \frac{1}{M} \nabla f(x).$$

这里每一步都将误差乘以固定因子 $\rho = 1 - m/M < 1$ 。

- ▶ 在二次函数或更强条件下，采用精确线搜索或最优固定步长时，常见速度可写成与条件数 $\kappa = M/m$ 有关的形式：

$$f(x^+) - f(x^*) \leq \left(\frac{\kappa - 1}{\kappa + 1}\right)^2 (f(x) - f(x^*)).$$

这仍然是线性收敛；平方只是说明收敛因子更小，不表示二次收敛。



更一般设置：光滑性给出什么？

- ▶ 假设：梯度的 Lipschitz 连续性：

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\| \quad \forall x, y$$

- ▶ 直观理解：梯度变化不会太突然，函数的弯曲程度由 L 控制。
- ▶ 推论（下降引理）：对任意 x, y ,

$$f(y) - f(x) \leq \nabla f(x)^T (y - x) + \frac{L}{2} \|y - x\|^2$$

- ▶ 它是分析梯度下降的基本工具：把一步下降量和步长 α 、梯度大小联系起来。



更一般设置：非凸光滑情形

- ▶ 若 f 下有界且 ∇f 为 L -Lipschitz, 采用常数步长 $0 < \alpha < 2/L$ 。
- ▶ 梯度下降可以保证：生成序列的任一聚点都是驻点。

$$x^{k_j} \rightarrow \bar{x} \implies \nabla f(\bar{x}) = 0.$$

- ▶ 但这不是全局最优保证。非凸函数中，驻点可能是局部极小点、鞍点或其他临界点。
- ▶ 因此非凸光滑情形的典型结论是：算法至少不会停在明显还能下降的一阶非驻点处。



更一般设置：凸但非强凸情形

- ▶ 如果进一步假设 f 是凸的，但不要求强凸，则可以得到函数值误差界。
- ▶ 对于常数步长 $0 < \alpha < 2/L$ 的梯度下降法，有

$$f(x^k) - f(x^*) \leq \frac{\|x^0 - x^*\|^2}{\alpha(2 - \alpha L)k}$$

- ▶ 因而

$$f(x^k) - f(x^*) = O(1/k),$$

这是次线性收敛，而不是线性收敛。

- ▶ 若要达到 $f(x^k) - f(x^*) \leq \epsilon$ ，通常需要 $k = O(1/\epsilon)$ 次迭代。



收敛性结论对比

- ▶ 光滑但非凸：主要看一阶驻点性。

$$\nabla f(\bar{x}) = 0$$

这不等于全局最优。

- ▶ 光滑凸但非强凸：看目标函数误差。

$$f(x^k) - f(x^*) = O(1/k), \quad k = O(1/\epsilon).$$

这是次线性收敛。

- ▶ 光滑强凸：目标函数误差按固定比例缩小。

$$f(x^k) - f(x^*) \leq \rho^k (f(x^0) - f(x^*)), \quad 0 < \rho < 1.$$

这是线性收敛，对应 $k = O(\log(1/\epsilon))$ 。



本章小结

- ▶ 推荐系统中的矩阵分解提供了大规模优化的典型应用背景。
- ▶ 下降方向决定是否降低目标函数，步长规则决定每一步降低多少。
- ▶ 光滑凸问题通常得到次线性收敛；光滑强凸问题可得到线性收敛，速度与条件数密切相关。
- ▶ 随机梯度下降用单个样本或小批量近似全梯度，是处理大规模稀疏数据的核心方法之一。